Task-1: Import Required Libraries

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Note:

1. **NumPy** is used for numerical computations and handling arrays.
2. **Pandas** is used for data manipulation and analysis (like reading CSV files, filtering, grouping).
3. **Matplotlib** is used for creating visualizations such as bar charts, line graphs, and histograms.

Task-2: Load Dataset & Preview

Load dataset

In [5]:
```
##Load dataset
df = pd.read_csv("sales.csv")
```

Dsiplay first 5 rows

In [6]:
```
df.head()
```

Out[6]:

	order_id	customer_id	
0	00010242fe8c5a6d1ba2dd792cb16214	3ce436f183e68e07877b285a838db11a	42447
1	00018f77f2f0320c557190d7a144bdd3	f6dd3ec061db4e3987629fe6b26e5cce	e5f2d
2	000229ec398224ef6ca0657da4fc703e	6489ae5e4333f3693df5ad4372dab6d3	c777
3	00024acbcdf0a6daa1e931b038114c75	d4eb9395c8c0431ee92fce09860c5a06	7634
4	00042b26cf59d7ce69dfabb4e55b4fd9	58dbd0b2d70206bf40e62cd34e84d795	ac6c3

Display last 5 rows

In [7]:
```
df.tail()
```

Out[7]:

	order_id	customer_id
117599	fffc94f6ce00a00581880bf54a75a037	b51593916b4b8e0d6f66f2ae24f2673d
117600	fffcd46ef2263f404302a634eb57f7eb	84c5d4fbaf120aae381fad077416eaa0
117601	fffce4705a9662cd70adb13d4a31832d	29309aa813182aaddc9b259e31b870e6
117602	fffe18544ffabc95dfada21779c9644f	b5e6afd5a41800fdf401e0272ca74655
117603	fffe41c64501cc87c801fd61db3f6244	96d649da0cc4ff33bb408b199d4c7dcf

Observation:

The dataset was successfully loaded into a Pandas DataFrame. Based on the preview of the first and last five rows:

- The data contains 117,604 entries across 11 columns.
- All columns are visible and appear to have correctly aligned headers and values.
- The dataset includes a variety of data types: categorical (e.g., category_english, payment_type), numerical (e.g., price, freight_value), and timestamps (currently in string format).

Task-3: Basic Info

Shape

In [8]:
```
df.shape
```

Out[8]:
```
(117604, 11)
```

The dataset contains 117604 rows and 11 columns

Info

In [9]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 117604 entries, 0 to 117603
Data columns (total 11 columns):
 #   Column                         Non-Null Count   Dtype
---  ------                         --------------   -----
 0   order_id                       117604 non-null  str
 1   customer_id                    117604 non-null  str
 2   product_id                     117604 non-null  str
 3   category_english               115881 non-null  str
 4   price                          117604 non-null  float64
 5   freight_value                  117604 non-null  float64
 6   payment_type                   117601 non-null  str
 7   payment_value                  117601 non-null  float64
 8   order_purchase_timestamp       117604 non-null  str
 9   order_delivered_customer_date  115037 non-null  str
 10  total_price                    117604 non-null  float64
dtypes: float64(4), str(7)
memory usage: 9.9 MB
```

Describe

In [10]:
```
df.describe()
```

Out[10]:

	price	freight_value	payment_value	total_price
count	117604.000000	117604.000000	117601.000000	117604.000000
mean	120.822849	20.045551	172.686752	140.868400
std	184.477367	15.861351	267.592290	191.616461
min	0.850000	0.000000	0.000000	6.080000
25%	39.900000	13.080000	60.870000	55.270000
50%	74.900000	16.290000	108.210000	92.115000
75%	134.900000	21.190000	189.260000	157.780000
max	6735.000000	409.680000	13664.080000	6929.310000

Observation:

The dataset contains 117,604 rows and 11 columns. While most features are complete, there are missing values in category_english, order_delivered_customer_date, and payment_type

---

Task-4: Identify the data types of all columns and convert date columns to the datetime format

Check datatypes

In [11]:
```
df.dtypes
```

Out[11]:
```
order_id                            str
customer_id                         str
product_id                          str
category_english                    str
price                           float64
freight_value                   float64
payment_type                        str
payment_value                   float64
order_purchase_timestamp            str
order_delivered_customer_date       str
total_price                     float64
dtype: object
```

Note:

Date related columns, such as order_purchase_timestamp and order_delivered_customer_date, are stored as str. Converting their dtypes to datetime is neccessary for calculating dalivery time and time-series analysis.

Convert date columns to datetime

In [14]:
```
df["order_purchase_timestamp"] = pd.to_datetime(df["order_purchase_timestamp"]
df["order_delivered_customer_date"] = pd.to_datetime(df["order_delivered_custo
```

In [15]:
```
df.dtypes
```

Out[15]:
```
order_id                                    str
customer_id                                 str
product_id                                  str
category_english                            str
price                                   float64
freight_value                           float64
payment_type                                str
payment_value                           float64
order_purchase_timestamp         datetime64[us]
order_delivered_customer_date    datetime64[us]
total_price                             float64
dtype: object
```

Note:

The datatyte of the date related columns is converted to datetime64

Task-5: Check for missing values and duplicates

Check missing values

In [16]:
```
df.isna().sum()
```

Out[16]:
```
order_id                           0
customer_id                        0
product_id                         0
category_english                1723
price                              0
freight_value                      0
payment_type                       3
payment_value                      3
order_purchase_timestamp           0
order_delivered_customer_date   2567
total_price                        0
dtype: int64
```

Observation:

category_english, payment_type, payment_valu and order_delivered_customer_date contain 1723, 3, 3 and 2567 missing records respectively.

We need to drop records that have missing values in "order_delivered_customer_date" column as it is required to calculate delivery time.

Handle missing values

In [17]:
```
df.dropna(subset=["order_delivered_customer_date"], inplace=True)
```

In [18]:
```
df.isna().sum()
```

Out[18]:
```
order_id                            0
customer_id                         0
product_id                          0
category_english                 1651
price                               0
freight_value                       0
payment_type                        3
payment_value                       3
order_purchase_timestamp            0
order_delivered_customer_date       0
total_price                         0
dtype: int64
```

Observation:

All rows having missing "*order_delivered_customer_date*" have been dropped.

Check duplicates

In [19]:
```
df.duplicated().sum()
```

Out[19]:
```
np.int64(10978)
```

Observation:

The dataset contains **10978** duplicate records.

Remove duplicates

In [20]:
```
df.drop_duplicates(inplace=True)
```

In [21]:
```
df.duplicated().sum()
```

Out[21]:
```
np.int64(0)
```

Observation:

All duplicate records have been removed.

Task-6: Use NumPy to calculate basic statistic

price

In [22]:
```
price_mean = np.mean(df["price"])
price_median = np.median(df["price"])
price_min = np.min(df["price"])
price_max = np.max(df["price"])
price_std = np.std(df["price"])

print("Price statistics:")
print("mean =",price_mean)
print("median =",price_median)
print("min value =",price_min)
print("max value =",price_max)
print("standard deviation value =",price_std)
```

```
Price statistics:
mean = 123.75611681834341
median = 78.0
min value = 0.85
max value = 6735.0
standard deviation value = 188.38384995690043
```

freight_value

In [23]:
```
freight_mean = np.mean(df["freight_value"])
freight_median = np.median(df["freight_value"])
freight_min = np.min(df["freight_value"])
freight_max = np.max(df["freight_value"])
freight_std = np.std(df["freight_value"])

print("Freight value statistics:")
print("mean =",freight_mean)
print("median =",freight_median)
print("min value =",freight_min)
print("max value =",freight_max)
print("standard deviation value =",freight_std)
```

```
Freight value statistics:
mean = 20.099157593288425
median = 16.34
min value = 0.0
max value = 409.68
standard deviation value = 15.805553823427182
```

Observation:

The analysis reveals a significant right-skew in both variables, as the means are

notably higher than the medians. Product prices show extreme variability, ranging from 0.85 to 6,735.00, with a high standard deviation indicating a diverse product catalog. Freight values are more consistent but include outliers up to 409.68, which likely represent bulky items or long-distance deliveries.

Task 7: Create new derived columns such as delivery_days and total_cost

Delivery time

In [64]:
```
df["delivery_time"] = (df["order_delivered_customer_date"] - df["order_purchas
```

Total cost (price + freight_value)

In [28]:
```
df["total_cost"] = df["price"] + df["freight_value"]
```

Preview new columns

In [65]:
```
df[["total_price", "delivery_time"]]
```

Out[65]:

	total_price	delivery_time
0	72.19	7
1	259.83	16
2	216.87	7
3	25.78	6
4	218.04	25
...	...	...
117599	343.40	17
117600	386.53	9
117601	116.85	4
117602	64.71	1
117603	55.79	5

104059 rows × 2 columns

Task-8: Extract date components

In [30]:
```
##Extract year
df["purchase_year"] = df["order_purchase_timestamp"].dt.year

#Extract month
df["purchase_month"] = df["order_purchase_timestamp"].dt.month

#Extract day
df["purchase_day"] = df["order_purchase_timestamp"].dt.day

#Extract day of week
df["purchase_day_of_week"] = df["order_purchase_timestamp"].dt.day_of_week

#Preview
df[["purchase_year", "purchase_month", "purchase_day", "purchase_day_of_week"]
```

Out[30]:

	purchase_year	purchase_month	purchase_day	purchase_day_of_week
0	2017	9	13	2
1	2017	4	26	2
2	2018	1	14	6
3	2018	8	8	2
4	2017	2	4	5
...	...	...	...	...
117599	2018	4	23	0
117600	2018	7	14	5
117601	2017	10	23	0
117602	2017	8	14	0
117603	2018	6	9	5

104059 rows × 4 columns

Observation:

- The purchase timestamp was broken down into multiple components:
  - Year → to analyze yearly trends
  - Month → to identify monthly sales patterns
  - Day → to observe daily variations

        - Day of week → to analyze weekday vs weekend behavior

- These new columns enable better time-based analysis and visualization in later steps.

Task 9: Display the top 5 categories by total sales revenue

Group by category and calculate total revenue

In [31]:
```
top_categories = (
    df.groupby("category_english")["total_price"]
    .sum()
    .sort_values(ascending=False)
    .head(5)
)
```

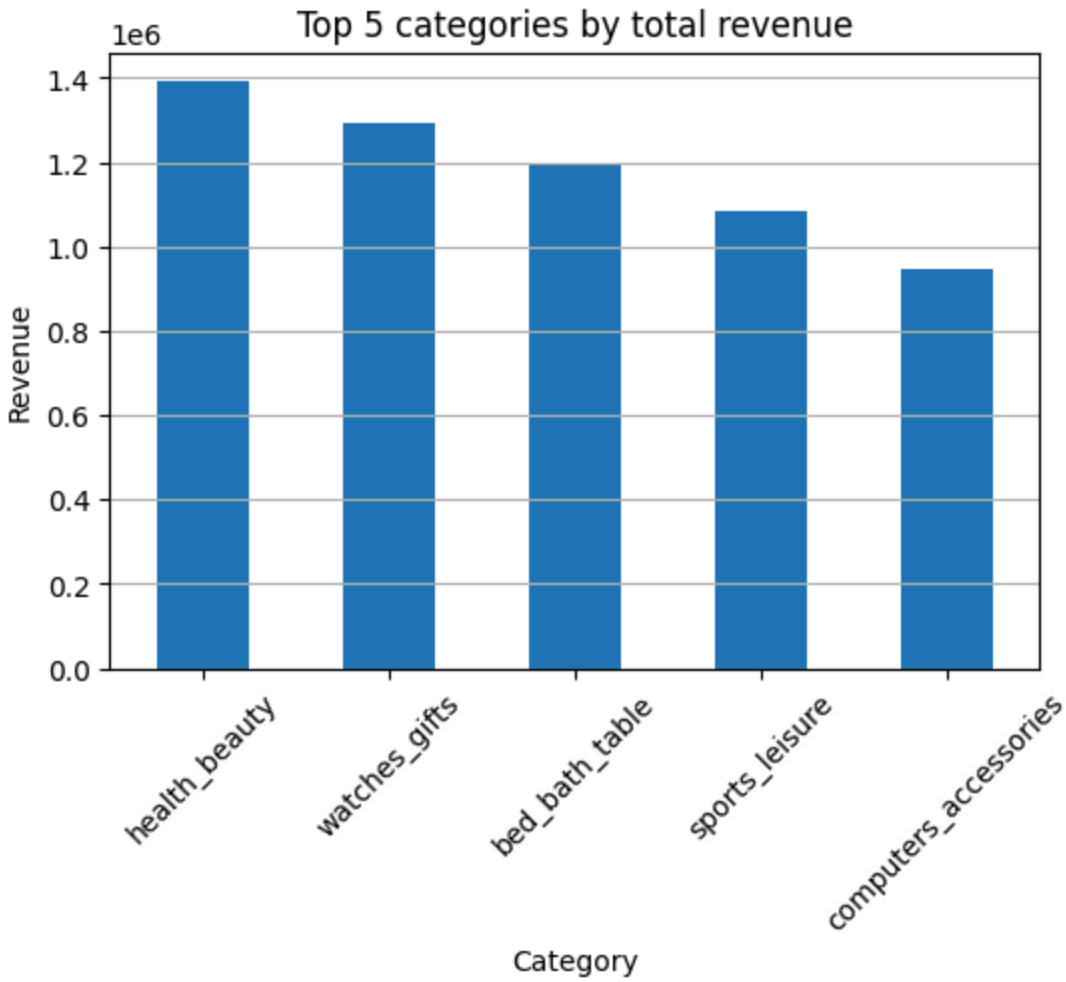
Display result

In [32]:
```
top_categories
```

Out[32]:
```
category_english
health_beauty            1391226.22
watches_gifts            1291280.10
bed_bath_table           1192640.35
sports_leisure           1082019.23
computers_accessories     944152.29
Name: total_price, dtype: float64
```

Visualization

In [44]:
```
plt.figure(figsize=(6, 4))
top_categories.plot(kind="bar")
plt.title("Top 5 categories by total revenue")
plt.xlabel("Category")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.grid(visible=True, axis="y")
plt.show()
```



Observation:

The analysis identifies *health_beauty*, *watches_gifts*, and *bed_bath_table* as the leading revenue drivers. The high performance of health and beauty products suggests a strong consumer focus on personal care, while the presence of watches and gifts indicates that the platform is a popular choice for high-ticket or seasonal gift shopping.

Task 10: Find the most common payment type and visualise its proportion using a bar chart

Count frequency of each payment type

In [39]:
```
payment_type_count = df["payment_type"].value_counts()
```

```
payment_type_count
```

Out[39]:
```
payment_type
credit_card    77538
boleto         19853
voucher         5120
debit_card      1547
Name: count, dtype: int64
```

Identify most common payment type

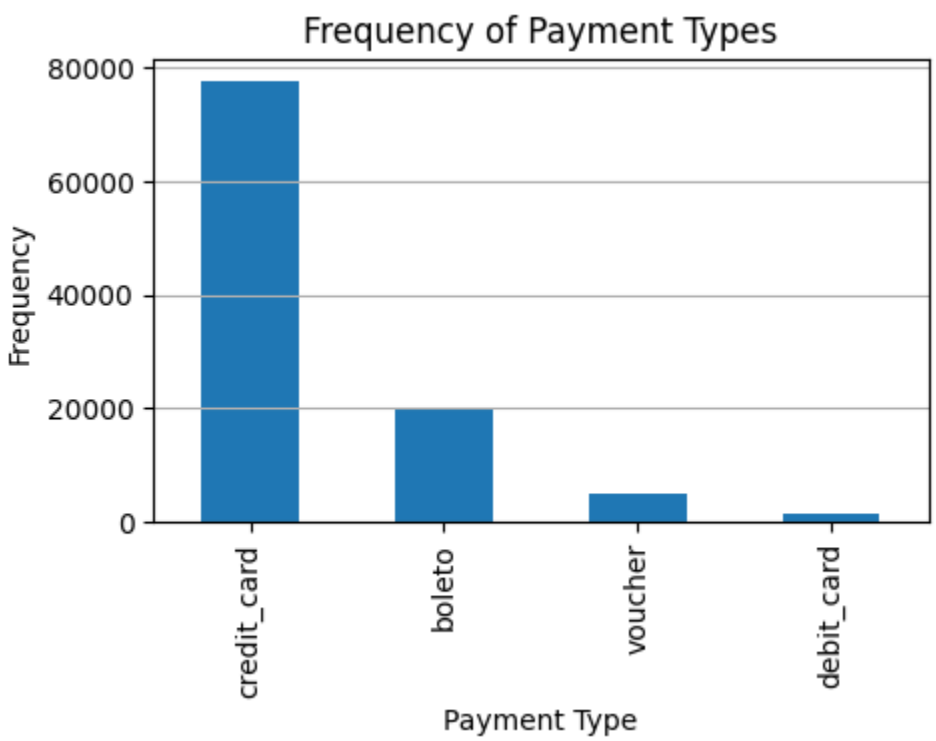
In [42]:
```
most_common_payment_type = payment_type_count.idxmax()

print("Most common payment type is:", most_common_payment_type)
```

```
Most common payment type is: credit_card
```

Visualization

In [47]:
```
plt.figure(figsize=(5, 3))
payment_type_count.plot(kind="bar")
plt.title("Frequency of Payment Types")
plt.xlabel("Payment Type")
plt.ylabel("Frequency")
plt.grid(visible=True, axis="y")
plt.show()
```



Observation:

In this dataset ***credit_card*** as the dominant payment method, accounting for the vast majority of transactions. This is followed by boleto, while voucher and debit_card represent a much smaller share of the total. The high prevalence of credit card usage suggests a customer base that prefers deferred payments or installment options.

Task 11: Calculate average delivery time per category using groupby

Average delivery time per category

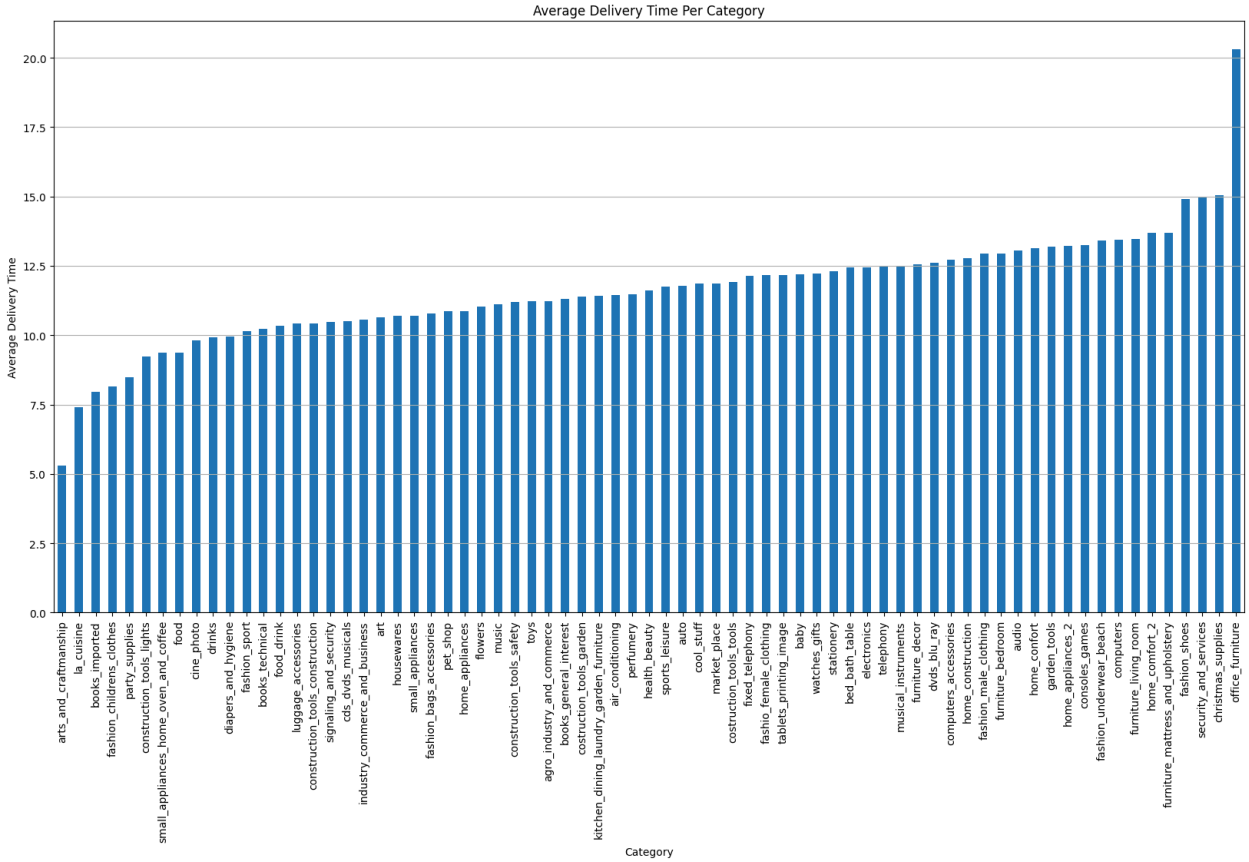
In [66]:
```
avg_delivery_time = df.groupby("category_english")["delivery_time"].mean().sor

avg_delivery_time
```

Out[66]:
```
category_english
arts_and_craftmanship                 5.291667
la_cuisine                            7.400000
books_imported                        7.965517
fashion_childrens_clothes             8.142857
party_supplies                        8.487179
                                        ...
furniture_mattress_and_upholstery    13.700000
fashion_shoes                        14.897727
security_and_services                15.000000
christmas_supplies                   15.046512
office_furniture                     20.307405
Name: delivery_time, Length: 71, dtype: float64
```

Visualization

In [68]:
```
avg_delivery_time.plot(
    kind="bar",
    figsize=(20, 10),
    title="Average Delivery Time Per Category",
    xlabel="Category",
    ylabel="Average Delivery Time",
)
plt.grid(visible=True, axis="y")
plt.show()
```



Observation:

We can see significant variation across different categories. Smaller items like **arts_and_craftmanship** and *books_imported* takes average 5 to 88 days to the delivered. However *office_furniture* takes the longest with average 20 days of delivery time.

Task 12: Analyse total monthly sales trends — use line charts to visualise growth over time

Monthly sales aggregation over the years

In [78]:
```
## Create Year-Month column
df["year_month"] = df["order_purchase_timestamp"].dt.to_period("M")

# Aggregate total sales per month
monthly_sales = df.groupby("year_month")["total_cost"].sum()

# Convert to string for plotting
```

```
monthly_sales.index = monthly_sales.index.astype(str)

monthly_sales
```

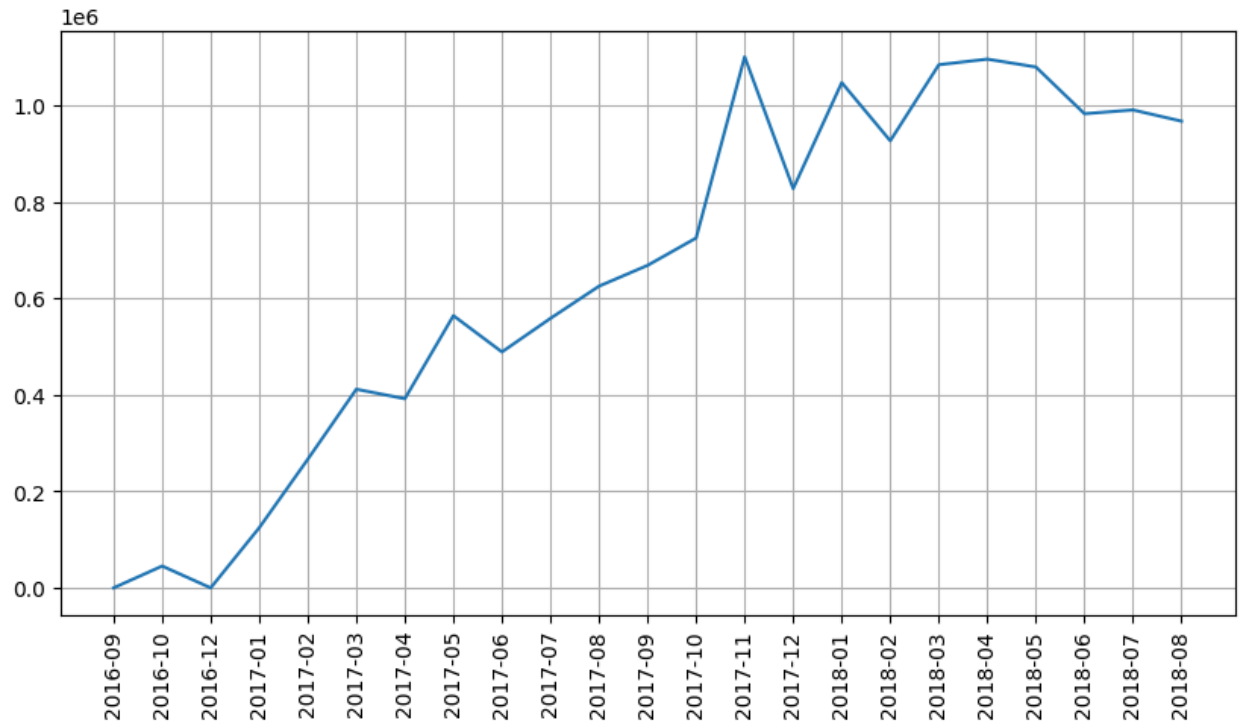
Out[78]:

```
year_month
2016-09         47.82
2016-10      45245.44
2016-12         19.62
2017-01     124640.85
2017-02     266522.65
2017-03     411643.39
2017-04     391986.71
2017-05     564074.96
2017-06     488889.26
2017-07     558463.51
2017-08     625151.92
2017-09     668250.16
2017-10     724922.61
2017-11    1100454.91
2017-12     827302.88
2018-01    1046921.56
2018-02     926375.10
2018-03    1083955.11
2018-04    1095574.83
2018-05    1079244.37
2018-06     982450.92
2018-07     990253.46
2018-08     967043.96
Name: total_cost, dtype: float64
```

Visualization

In [79]:

```
plt.figure(figsize=(10, 5))
plt.plot(monthly_sales.index, monthly_sales.values)
plt.xticks(rotation=90)
plt.grid(visible=True)
plt.show()
```



Observation:

- We can observe a clear upward trajectory in 2017 with a massive revenue spike in November, 2017 (surpassing 1.1M).
- In late 2017 to mid 2018 we can observe significant volatility.

Total monthly sales

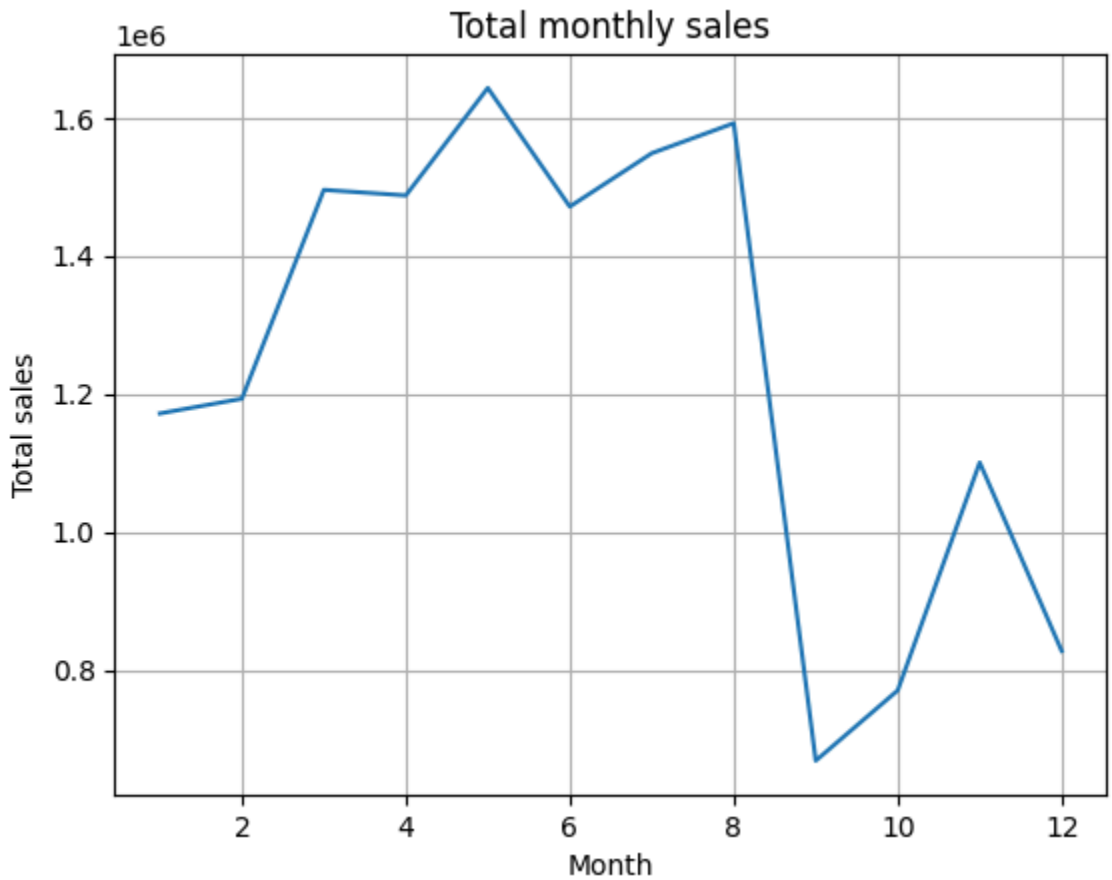
In [80]:
```
total_monthly_sales = df.groupby("purchase_month")["total_cost"].sum()

total_monthly_sales
```

Out[80]:
```
purchase_month
1     1171562.41
2     1192897.75
3     1495598.50
4     1487561.54
5     1643319.33
6     1471340.18
7     1548716.97
8     1592195.88
9      668297.98
10     770168.05
11    1100454.91
12     827322.50
Name: total_cost, dtype: float64
```

Visualization

In [82]:
```
total_monthly_sales.plot(
    title="Total monthly sales",
    xlabel="Month",
    ylabel="Total sales",
    grid=True,
)
plt.show()
```



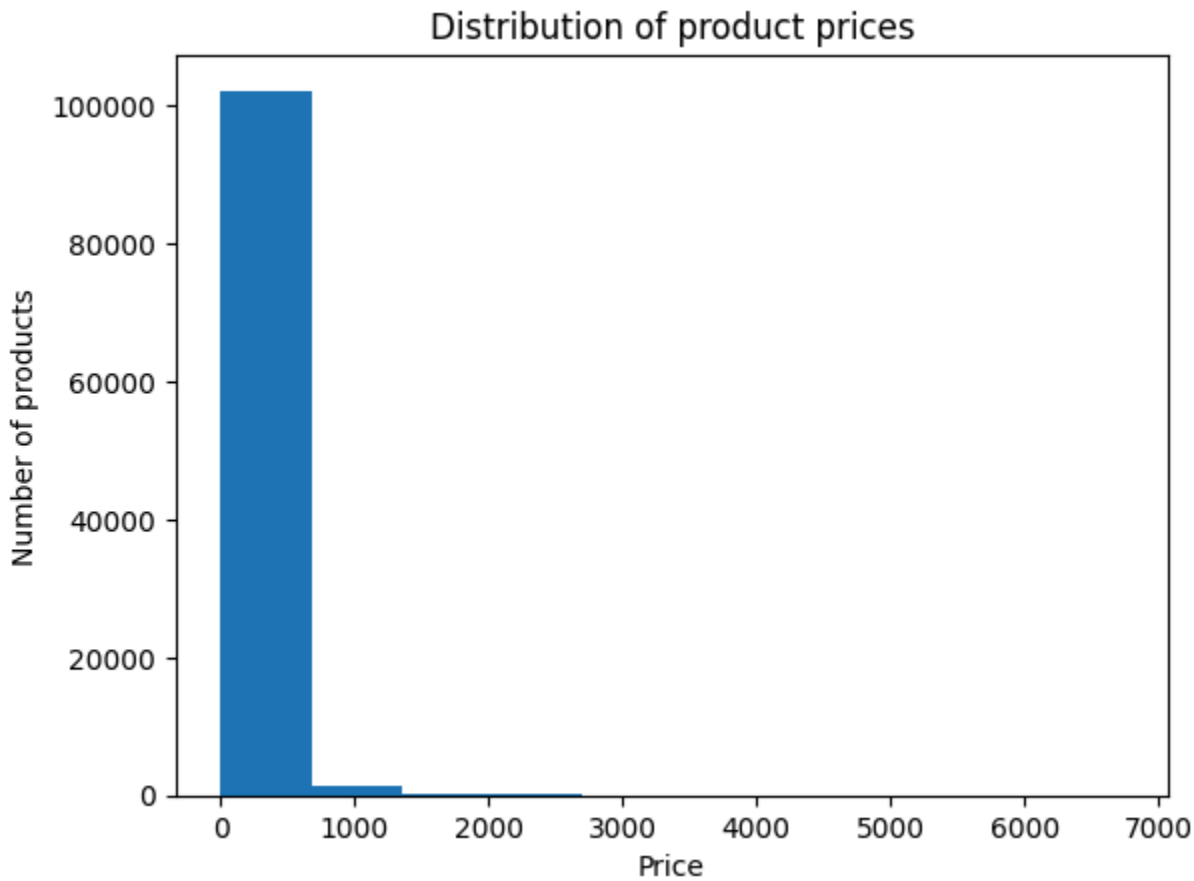
Observation

By aggregating data by month, a clear seasonal pattern emerges. Sales are strongest during the first half of the year, peaking in May, while there is a sharp, unexpected decline in September and October.

Task 13: Create a histogram showing the distribution of product prices

Histogram of product prices

In [83]:
```
plt.hist(df["price"], bins=10)
plt.title("Distribution of product prices")
plt.xlabel("Price")
plt.ylabel("Number of products")
plt.show()
```



Histogram plot of a smaller range (0-1000)

In [84]:
```
plt.hist(df["price"], bins=70)

plt.xlim(0, 1000)
plt.xticks(np.arange(0, 1001, 100), rotation=90)
plt.grid(visible=True)

plt.title("Distribution of product prices")
plt.xlabel("Price")
plt.ylabel("Number of products")
```

```
plt.show()
```

Observation:

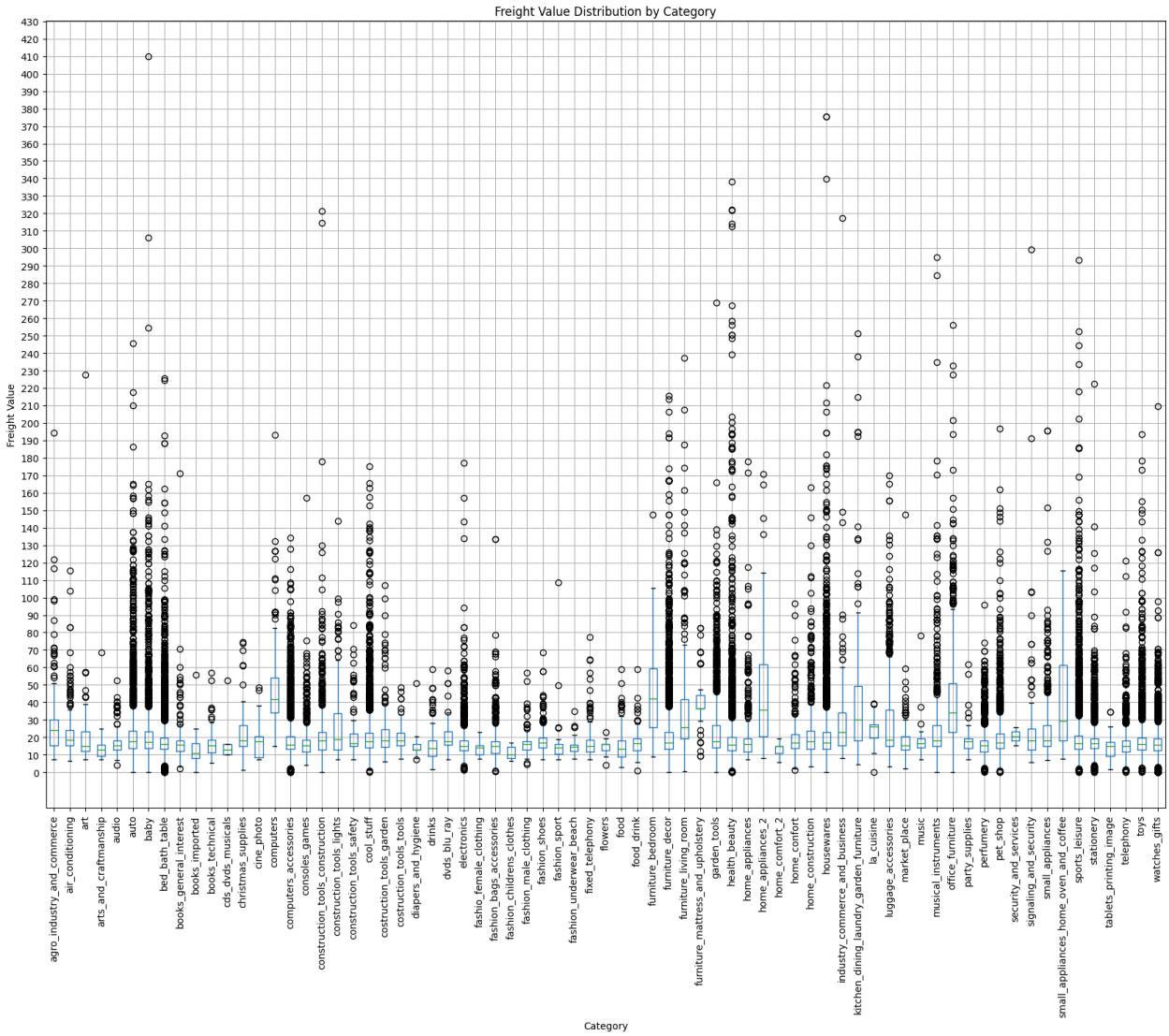
The histogram reveals a highly skewed distribution, with the vast majority of products priced under $500. As the price increases, the frequency drops sharply, indicating that the marketplace primarily thrives on high-volume, affordable consumer goods. The long tail extending toward the $6,000+ range represents a very small fraction of premium or luxury items, which contribute to the high standard deviation observed in earlier statistical tasks.

Task 14: Use a box plot to compare freight_value across different categories

In [85]:
```
df.boxplot(column="freight_value", by="category_english", rot=90, figsize=(20,
plt.yticks(np.arange(0, 431, 10))

plt.title("Freight Value Distribution by Category")
```

```
plt.suptitle("")  # removes default extra title
plt.xlabel("Category")
plt.ylabel("Freight Value")
plt.show()
```



Observation:

- The boxplot shows significant variability in shipping cost across categories.
- While most categories show low and tight Interquertile Range (IQR), certain sectors like office_furniture, fixed_telephony, and construction_tools_heavy exhibit much higher median freight values and a wider spread.
- We can also observe numerous outliers aqcross categories. It signifies other than categories shipping cost must alos depend on other factors

like *volumetric weight* and *geographical distance*

Task 15: Identify the top 10 customers by total spending and visualise as a horizontal bar chart

Top 10 customers by total spending

In [86]:
```
top_paying_customers = df.groupby("customer_id")["total_price"].sum().sort_val

top_paying_customers
```

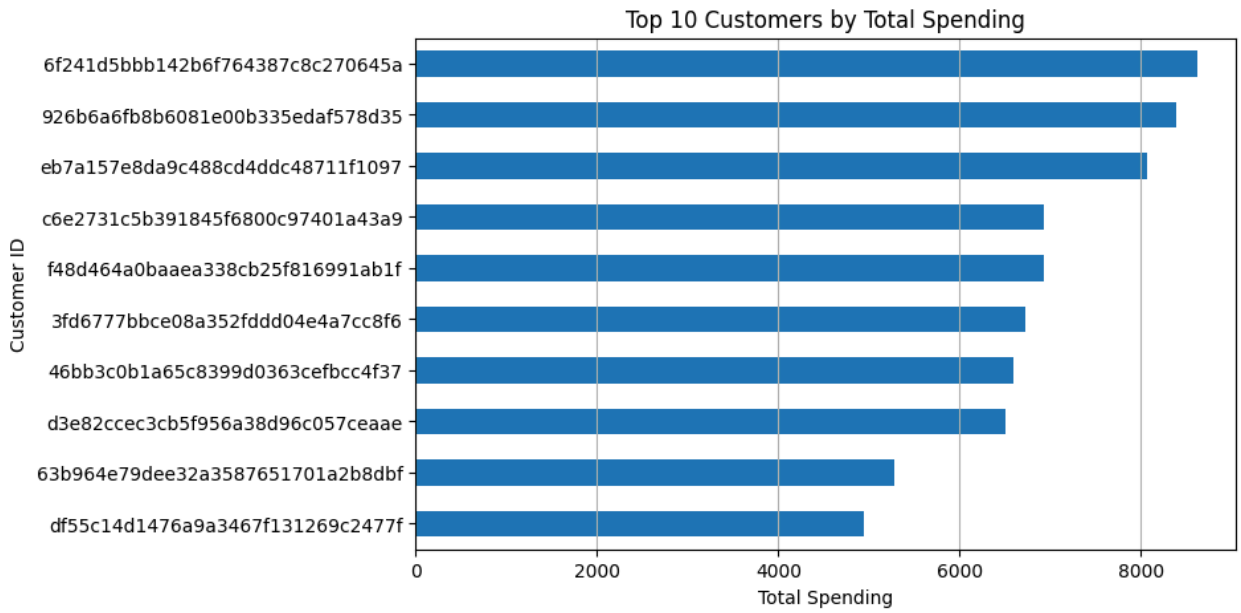
Out[86]:
```
customer_id
6f241d5bbb142b6f764387c8c270645a    8618.76
926b6a6fb8b6081e00b335edaf578d35    8389.52
eb7a157e8da9c488cd4ddc48711f1097    8068.88
c6e2731c5b391845f6800c97401a43a9    6929.31
f48d464a0baaea338cb25f816991ab1f    6922.21
3fd6777bbce08a352fddd04e4a7cc8f6    6726.66
46bb3c0b1a65c8399d0363cefbcc4f37    6594.80
d3e82ccec3cb5f956a38d96c057ceaae    6511.80
63b964e79dee32a3587651701a2b8dbf    5276.64
df55c14d1476a9a3467f131269c2477f    4950.34
Name: total_price, dtype: float64
```

Horizontal bar chart

In [87]:
```
plt.figure(figsize=(8, 5))
top_paying_customers.sort_values().plot(kind="barh")
plt.grid(visible=True, axis="x")

plt.title("Top 10 Customers by Total Spending")
plt.xlabel("Total Spending")
plt.ylabel("Customer ID")

plt.show()
```



Observation:

- The above plot clearly shows top 10 customers in terms of total spending.
- Where 3 high value customers spending over $8000, 5 of them spending between $6500 and $7000 and the rest two spends around $5000.
- They are likely power user or professional buyer.

Task 16: Calculate the correlation between price, freight_value, and total_price. Visualise correlation using a heatmap.

Calculate correlation

In [88]:
```
corr_matrix = df[["price", "freight_value", "total_price"]].corr()

corr_matrix
```

Out[88]:

	price	freight_value	total_price
price	1.000000	0.420269	0.997307
freight_value	0.420269	1.000000	0.485686
total_price	0.997307	0.485686	1.000000

Visualise correlation heatmap using seaborn library

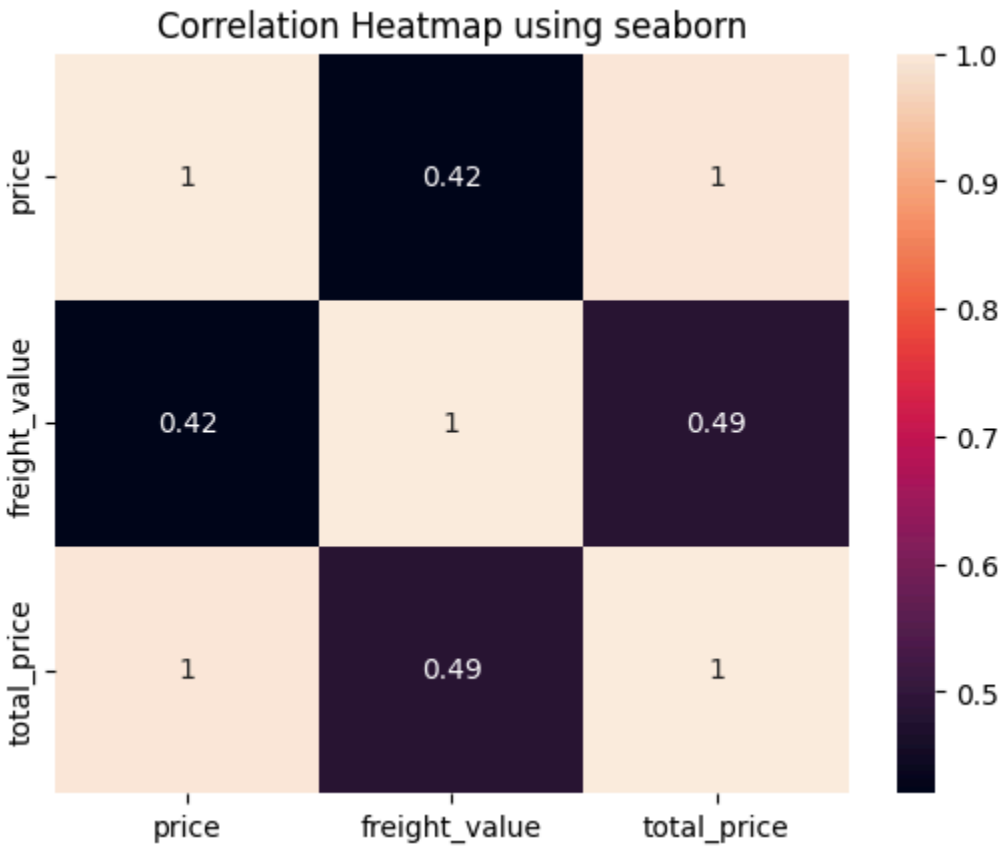
In [90]:
```
%pip install seaborn
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in C:\Users\User\AppData\Roaming\Python\
Python313\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\User\AppData\Ro
aming\Python\Python313\site-packages (from seaborn) (2.2.3)
Requirement already satisfied: pandas>=1.2 in C:\Users\User\AppData\Roaming\Pyt
hon\Python313\site-packages (from seaborn) (3.0.1)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\User\AppDat
a\Roaming\Python\Python313\site-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\User\AppData\Roamin
g\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(1.3.1)
Requirement already satisfied: cycler>=0.10 in C:\Users\User\AppData\Roaming\Py
thon\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\User\AppData\Roami
ng\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.5
6.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\User\AppData\Roami
ng\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(1.4.8)
Requirement already satisfied: packaging>=20.0 in C:\Users\User\AppData\Roamin
g\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.2)
Requirement already satisfied: pillow>=8 in C:\Users\User\AppData\Roaming\Pytho
n\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\User\AppData\Roamin
g\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\User\AppData\Ro
aming\Python\Python313\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\User\AppData\Roaming\Python\P
ython313\site-packages (from pandas>=1.2->seaborn) (2025.3)
Requirement already satisfied: six>=1.5 in C:\Users\User\AppData\Roaming\Pytho
n\Python313\site-packages (from python-dateutil>=2.7->matplotli
b!=3.6.1,>=3.4->seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [91]:
```
import seaborn as sns

sns.heatmap(corr_matrix, annot=True)
```

```
plt.title("Correlation Heatmap using seaborn")
plt.show()
```



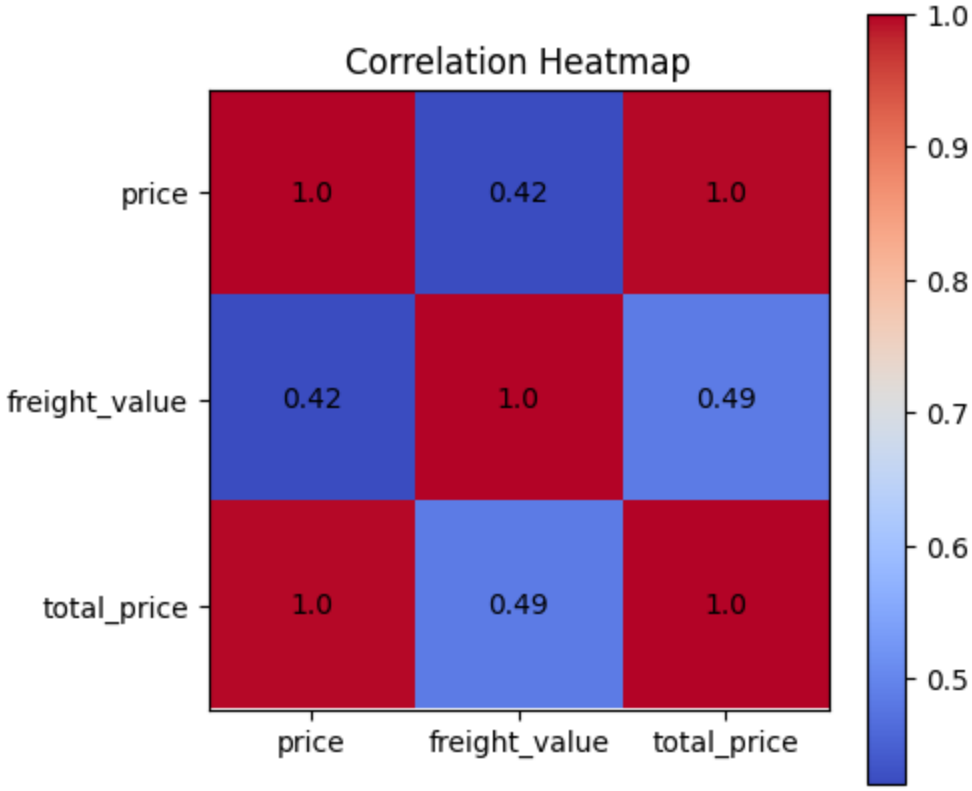
Visualise correlation heatmap using matplotlib library

In [92]:
```
plt.figure(figsize=(5, 5))
plt.imshow(corr_matrix, cmap="coolwarm", interpolation="none")
plt.colorbar()

plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns)
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        plt.text(i, j, round(corr_matrix.iloc[i, j], 2), ha="center", va="cent

plt.title("Correlation Heatmap")
plt.show()
```



Observation:

- The heatmap reveals an almost perfect positive correlation (0.997) between price and total_price, confirming that the base product cost is the primary driver of the final order value.
- The correlation between price and freight_value is relatively low (0.42), indicating that expensive items do not always incur high shipping costs.
- This suggests that shipping fees are likely more dependent on physical dimensions or weight rather than the monetary value of the product.

Task 17: Find how payment types vary by category — create a grouped bar plot

Payment type variation by category

In [94]:
```
payment_type_variation = pd.crosstab(df["category_english"], df["payment_type"

payment_type_variation
```

Out[94]:

payment_type category_english	boleto	credit_card	debit_card	voucher
agro_industry_and_commerce	49	123	4	28
air_conditioning	55	186	3	7
art	46	141	5	13
arts_and_craftmanship	7	14	3	0
audio	78	253	8	23
...	...	...	...	...
stationery	419	1847	36	98
tablets_printing_image	17	62	0	4
telephony	940	3117	77	184
toys	686	3125	43	168
watches_gifts	934	4626	61	224

71 rows × 4 columns

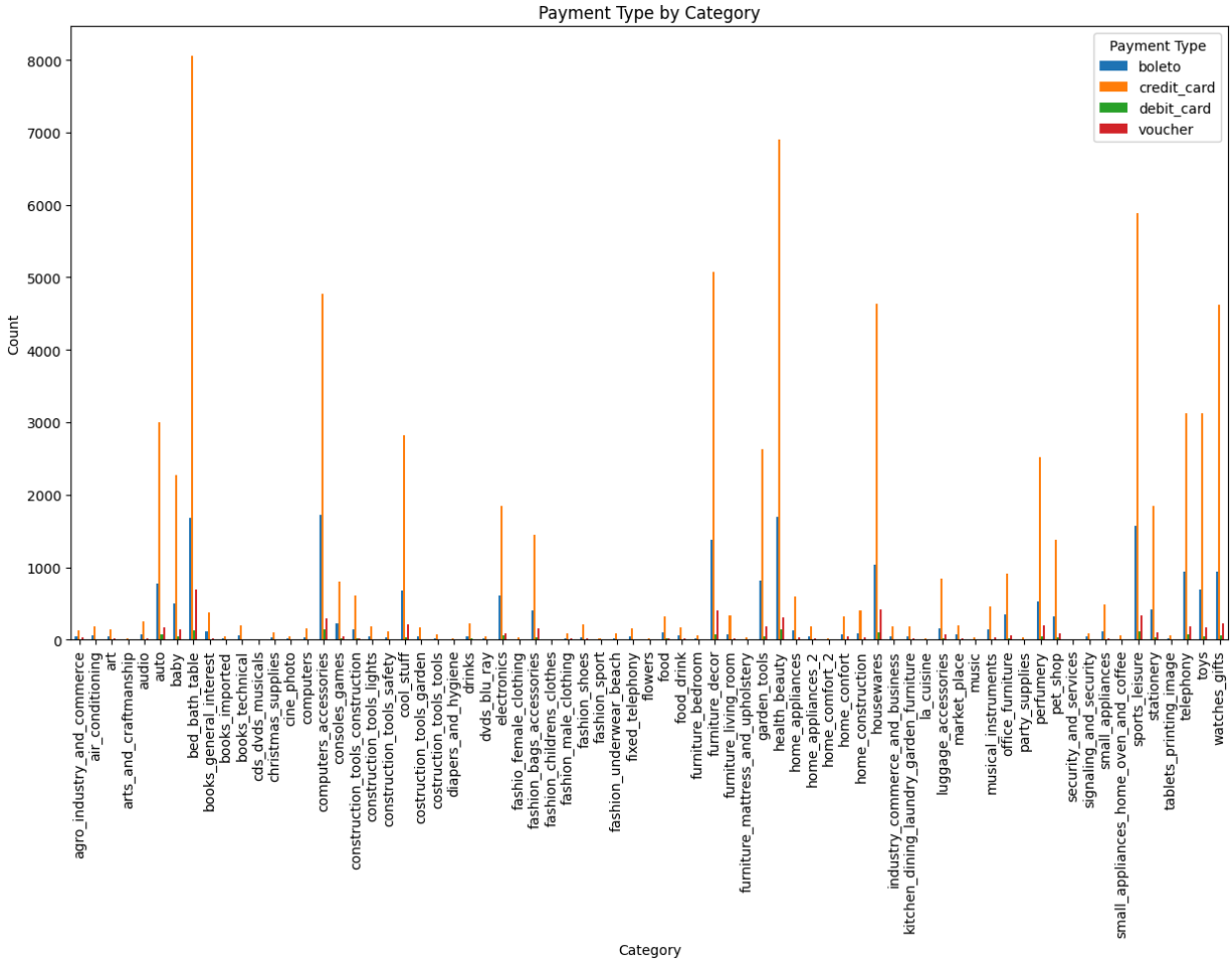
Grouped bar plot

In [95]:

```
payment_type_variation.plot(kind="bar", figsize=(15, 8))

plt.title("Payment Type by Category")
plt.xlabel("Category")
plt.ylabel("Count")

plt.legend(title="Payment Type")
plt.show()
```



Observation:

- We can observe that credit card is the most preferred payment method across most product categories.
- Interestingly, boleto (bank slips) maintain a consistent second-place presence across all categories, suggesting a specific segment of the customer base prefers cash-based or non-digital credit options regardless of what they are purchasing.

Task 18: Using NumPy, create an array of total_price values and compute its percentile distribution (25th, 50th, 75th)

Percentile calculation using NumPy

In [96]:
```
total_price_arr = np.array(df["total_price"])

p25 = np.percentile(total_price_arr, 25)
p50 = np.percentile(total_price_arr, 50)
p75 = np.percentile(total_price_arr, 75)

print("25th percentile:", p25)
print("50th percentile (Median):", p50)
print("75th percentile:", p75)
```

```
25th percentile: 56.885000000000005
50th percentile (Median): 94.7
75th percentile: 160.79
```

Observation:

The percentile analysis provides a clear breakdown of the order value distribution. Half of all transactions (the median) fall below $\$94.70$, while 75% of orders are priced under $\$160.79$. The significant gap between the 25th percentile ($\$56.89$) and the top end of the scale suggests that the business relies heavily on a high volume of mid-to-low-range purchases, with only a small minority of orders reaching the higher price brackets.

Task 19: Display orders with exceptionally high total_price (e.g., above 95th percentile)

Find 95th percentile threshold

In [97]:
```
threshold = np.percentile(df["total_price"], 95)

print(threshold)
```

```
388.38199999999983
```

Filter high-value orders

In [98]:
```
high_value_orders = df[df["total_price"] > threshold]

high_value_orders
```

Out[98]:

	order_id	customer_id
7	000576fe39319847cbb9d288c5617fa6	9ed5e522dd9dd85b4af4a077526d8117
16	0009c9a17f916a706d71784483a5d643	8a250edc40ebc5c3940ebc940f16a7eb
30	00137e170939bba5a3134e2386413108	7b63cba66b8e6d002e94d0990c1e2868
40	0017afd5076e074a48f1f1a4c7bac9c5	8085a9af46f619bc25966f151a362b0d
62	0025c5d1a8ca53a240ec2634bb4492ea	2430ad4b1b6efb56cf3050b5d3cf5e54
...	...	...
117506	ffc2638415f3ce34e88641eef792c1fc	cfd66b47b20e40dc4aef3c309508604e
117510	ffc49b9c9d666410e9ecdb0a3efadb9c	e175dcb4b025365a33467693abf7b4ca
117525	ffd5e80bfdaf69504efd7562a3d522dd	467bcdf6e97d661f3947ad9308dcb0cc
117537	ffdab5e1596138d37793566aa1424d0c	58f5f08f888dddeeb13e21d686545c6d
117586	fff7c4452f050315db1b3f24d9df5fcd	dec8952e97ef6124259c56914fb3569c

5203 rows × 18 columns

Observation:

- The 95th percentile was calculated to identify high-value transactions.
- Orders with total cost above this threshold were filtered as exceptionally high-value orders.
- These orders represent the top 5% of transactions in terms of spending.
- Such transactions may indicate:
    - Premium customers
    - Bulk purchases
    - Potential outliers

Task 20: Summarise key insights

1. Sales are growing, with a massive peak in November: Since late 2017, the store has been seeing more and more orders. The biggest spike happened in November. Generally, the first half of the year stays busier than the end of the year.

2. Credit cards are the most popular way to pay: Most customers prefer using credit cards for their purchases. Bank slips (boleto) are the second choice, but almost nobody uses debit cards or vouchers.

3. Health, beauty, and home goods make the most money: These categories are the real "bread and butter" of the store. They bring in the highest revenue and should probably be the main focus for ads and promotions.

4. Big items take a long time to arrive: There is a big delivery gap. While small things like books arrive quickly, heavy items like office furniture can take over 20 days to reach the customer. This is definitely something that needs to be fixed to keep customers happy.

5. Most customers are budget-conscious: Half of all orders are under $95, and the vast majority of products cost less than $500. While we do have a few very expensive luxury items, the store mostly survives on high volumes of affordable products.

Task 21: Identify the top 3 product categories that contribute the most to total revenue

Find top 3 categories by total revenue

In [104…

```
top3_categories = df.groupby("category_english")["total_price"].sum().sort_val

top3_categories
```

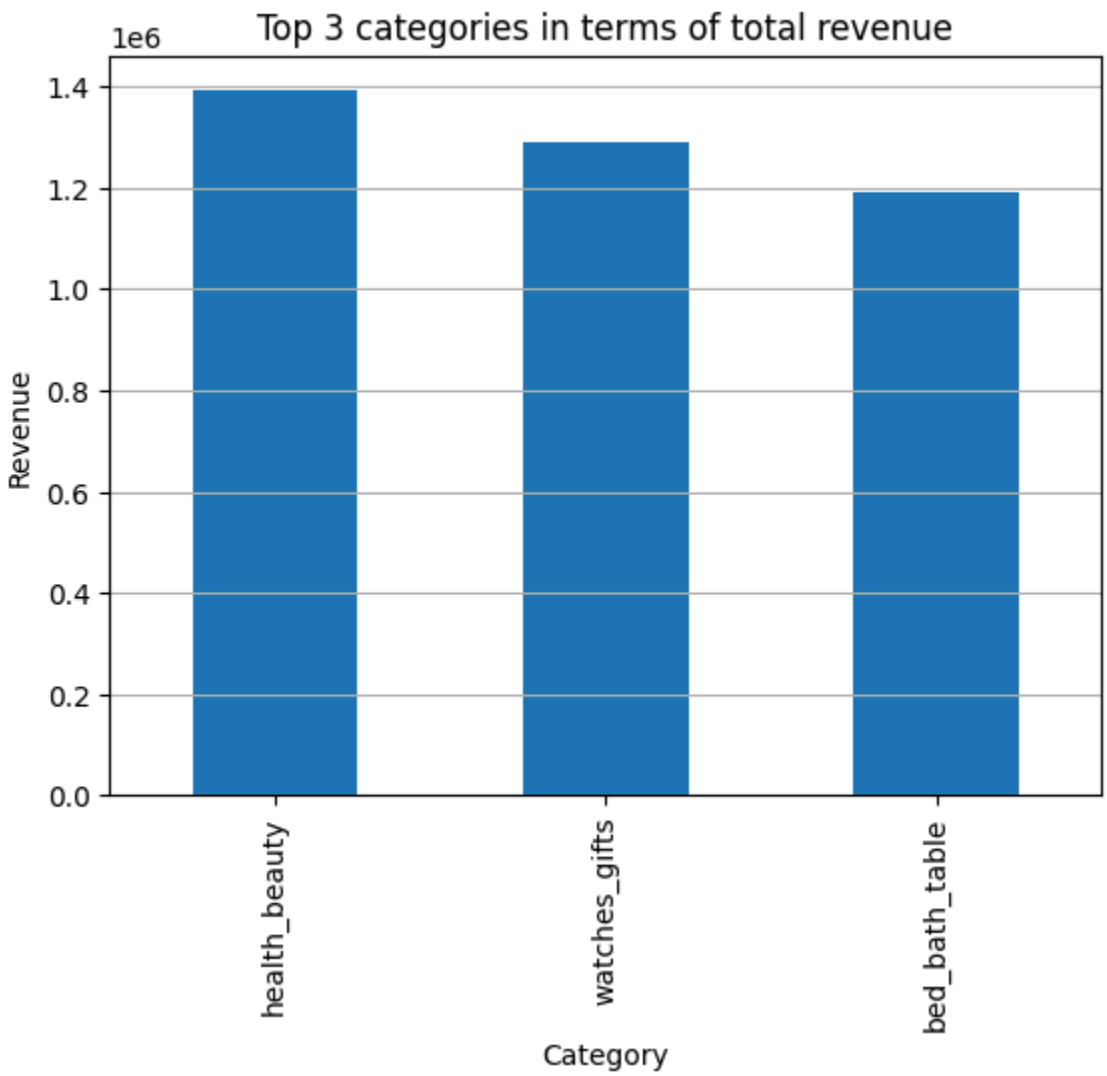
Out[104…

```
category_english
health_beauty     1391226.22
watches_gifts     1291280.10
bed_bath_table    1192640.35
Name: total_price, dtype: float64
```

Visualisation using bar chart

In [111…

```
top3_categories.plot(
    kind="bar",
    # figsize=(8, 5),
    title="Top 3 categories in terms of total revenue",
    xlabel="Category",
    ylabel="Revenue"
)
plt.grid(axis="y")
plt.show()
```



Observation:

We can clearly see that **health_beauty**, **watches_gifts** and **bed_bath_table** are the top 3 product categories contributing the most in the total revenue.

Task 22: Analyse if high shipping costs (freight_value) correlate with delayed deliveries

Overall correlation freight_value and delivery_time

In [114…

```
overall_corr = df["freight_value"].corr(df["delivery_time"])

print("Overall correlation freight_value and delivery_time:", overall_corr)
```

```
Overall correlation freight_value and delivery_time: 0.21011670399708346
```

Correlation for high shipping cost subset

In [ ]:

```
# Calculate high shipping cost threshold
high_shipping_threshold = np.percentile(df["freight_value"], 75)
print("high_shipping_threshold:", high_shipping_threshold)

# Filter out high shipping costs
high_shipping_records = df[df["freight_value"] > high_shipping_threshold]

# Calculate correlation
high_shipping_corr = high_shipping_records["freight_value"].corr(high_shipping

print("high_shipping_corr:", high_shipping_corr)
```

```
high_shipping_threshold: 21.22
high_shipping_corr: 0.059914075108271715
```

Observation:

- There is no strong link between shipping cost and delivery time.
- The overall correlation is quite low (0.21).
- After filtering out expensive shipping (75th percentile) and correlation with delivery time further dropped.
- This suggest paying high shipping cost does not mean getting faster delivery.

Task 23: Find which months have the highest number of orders and discuss possible reasons (e.g., seasonal sales)

Orders per month

In [117…

```
orders_per_month = df["purchase_month"].value_counts()

orders_per_month
```

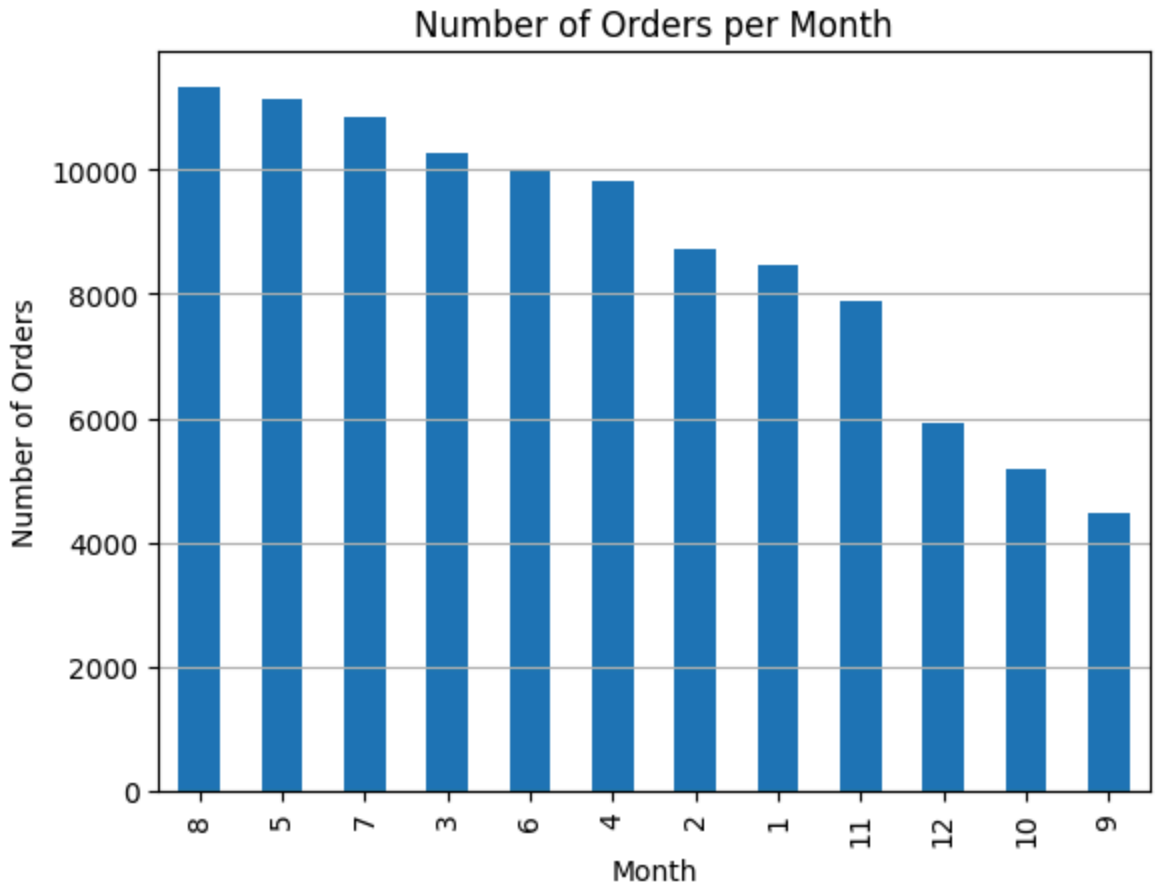
Out[117…

```
purchase_month
8     11328
5     11124
7     10832
3     10269
6      9986
4      9829
2      8727
1      8466
11     7891
12     5936
10     5183
9      4488
Name: count, dtype: int64
```

Visualisation

In [121…

```
orders_per_month.plot(
    kind="bar",
    title="Number of Orders per Month",
    xlabel="Month",
    ylabel="Number of Orders"
)
plt.grid(axis="y")
plt.show()
```



Observation:

The bar chart shows that ***August***, ***May***, and ***July*** had the most number of orders (over 10,500). This high volume of sales in mid-year suggests **"summer sales"** or **seasonal household shopping**.

Interestingly, while **November** doesn't have the highest count of orders, our previous revenue analysis showed it had the highest total sales value in 2017. This means that while people buy more frequently in the summer, they tend to spend much larger amounts on individual items during the November Black Friday period.

Task 24: Compare average order value (AOV) between different payment types

Average Order Value by payment type

In [122…

```
avg_order_value = (
    df.groupby("payment_type")["total_cost"]
    .mean()
```

```
    .sort_values(ascending=False)
)

avg_order_value
```

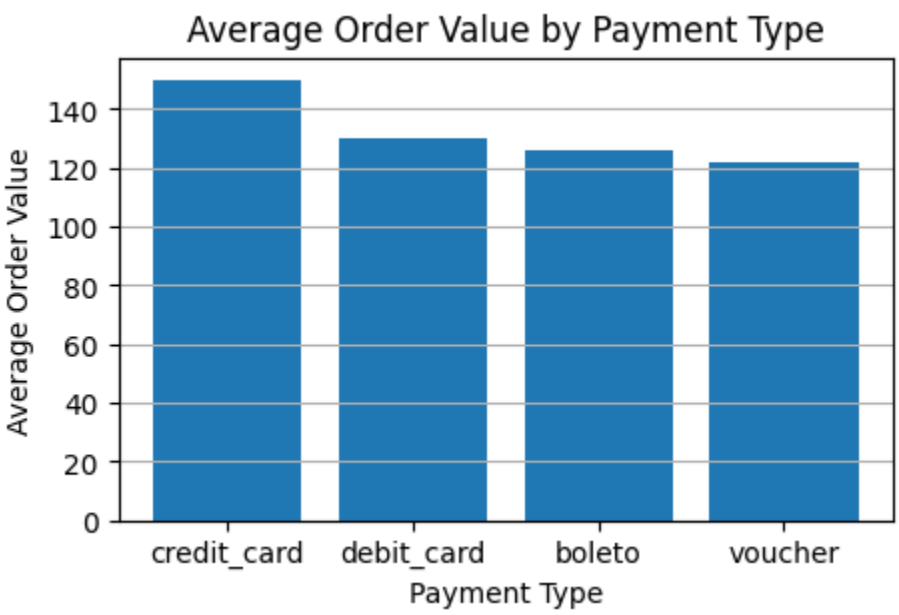
Out[122…

```
payment_type
credit_card    150.093467
debit_card     129.916341
boleto         126.210803
voucher        122.030721
Name: total_cost, dtype: float64
```

Visualisation

In [129…

```
plt.figure(figsize=(5, 3))
plt.bar(avg_order_value.index, avg_order_value.values)
plt.title("Average Order Value by Payment Type")
plt.xlabel("Payment Type")
plt.ylabel("Average Order Value")
plt.grid(axis="y")
plt.show()
```



Observation:

- We can clearly see that credit card carries the highest average order value (approximately $\$150.09$).
- Debit card and Boleto sits in the midle.
- Vouchers has the least avrage order value.

Task 25: Analyse customer purchase frequency

Purchase frequency for each customer

In [ ]:
```
order_count = df.groupby("customer_id")["order_id"].nunique()

order_count
```

Out[ ]:
```
customer_id
00012a2ce6f8dcda20d059ce98491703    1
000161a058600d5901f007fab4c27140    1
0001fd6190edaaf884bcaf3d49edf079    1
0002414f95344307404f0ace7a26f1d5    1
000379cdec625522490c315e70c7a9fb    1
                                   ..
fffcb937e9dd47a13f05ecb8290f4d3e    1
fffecc9f79fd8c764f843e9951b11341    1
fffeda5b6d849fbd39689bb92087f431    1
ffff42319e9b2d713724ae527742af25    1
ffffa3172527f765de70084a7e53aae8    1
Name: order_id, Length: 96476, dtype: int64
```

Top 5 repeat customers

In [133…
```
top5_repeat_customers = order_count.sort_values(ascending=False).head(5)

top5_repeat_customers
```

Out[133…
```
customer_id
ffffa3172527f765de70084a7e53aae8    1
00012a2ce6f8dcda20d059ce98491703    1
000161a058600d5901f007fab4c27140    1
0001fd6190edaaf884bcaf3d49edf079    1
0002414f95344307404f0ace7a26f1d5    1
Name: order_id, dtype: int64
```

Observation:

The output shows that the highest number of unique orders for any ***customer_id*** is 1. This indicates that either there are no repeat customers in this specific dataset, or the system generates a new, distinct customer_id for every single transaction, making it impossible to identify returning buyers using the available data

---